

# Eldermoor

Chronicles of the Hollow Vale

Technical Documentation and User Manual

Interactive Graphics Course Project

Three.js, WebGL, JavaScript

DENISE MARINI 1982720

## 1 Project Overview

**Eldermoor** is an interactive real-time 3D scene implemented with Three.js and JavaScript. The project can be understood as a small demo of a larger fantasy-medieval game: it presents an explorable village set in a valley, where a playable elven character walks through the streets, talks to townsfolk, gathers herbs, collects items and roams among houses, a working windmill, a well, market stalls, and cultivated fields, while a continuous day/night cycle changes the lighting of the world.

The application is designed as a complete browser-based graphics project: it has a clear theme, a navigable 3D world, animated hierarchical models, several light types with shadows, image-based and procedural textures, user controls and a small set of gameplay systems (dialogue, quests, inventory, collectibles).

### Main goals

- Build an open medieval environment that can be freely explored in third person.
- Use hierarchical modeling for articulated and composite objects such as the character skeleton, the windmill and the well.
- Show real-time rendering concepts such as meshes, materials, textures, lights, shadows and perspective camera projection.
- Implement character and world animation in JavaScript through a procedural animation system.
- Provide enough interaction for the user to navigate the world and engage with its gameplay systems.
- Keep the scene responsive across hardware through an adaptive quality system that detects the GPU and scales rendering, shadows and vegetation accordingly.

### Requirement coverage summary

| Requirement         | Where it appears in the project                                                                                                                                                                                                                                   |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Hierarchical models | The elven character and the NPCs are skinned meshes driven by a bone hierarchy; the windmill, well, market stalls, banners and lamp posts are built from nested <code>THREE.Group</code> objects.                                                                 |
| Lights and textures | Hemisphere and directional lights with soft shadows, plus a dynamic day/night cycle and emissive lamp posts that switch on at dusk. Materials combine loaded image textures (base colour, roughness, normal) with procedural <code>CanvasTexture</code> surfaces. |
| User interaction    | Third-person character control, a custom mouse-orbit follow camera with scroll zoom, proximity-based NPC dialogue, herb gathering, an openable inventory and a quest system.                                                                                      |

| Requirement   | Where it appears in the project                                                                                                                                                                                                                 |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Animations    | A procedural walk cycle, idle/rest pose and jump for the character; autonomous NPC wandering with gesture animation; rotating windmill sails; animated water; and the continuous day/night lighting transition — all implemented in JavaScript. |
| Documentation | This document describes the environment, libraries, technical implementation, interactions, how to run and the project scope.                                                                                                                   |

## 2 Environment, Libraries and External Assets

The project runs in a modern browser and uses WebGL through Three.js. The scene is rendered inside an HTML canvas. The main menu, loading screen and HUD are handled with HTML and CSS, while all 3D logic is implemented in JavaScript modules.

### Environment

- **HTML:** page shell, canvas element, main menu, animated loading screen and HUD container.
- **CSS:** full-screen layout, menu and loading overlays, medieval-styled UI and progress bar.
- **JavaScript modules:** scene creation, geometry construction, model loading, texture and material setup, lights, animation loop, user input and gameplay systems.
- **Three.js r160:** WebGL renderer, scene graph, cameras, lights, shadows, geometry, materials, textures and math utilities.

### Libraries and addons not developed by the author

| Library or addon                                         | Use in the project                                                                                                                              |
|----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>three.module.js</code>                             | Main Three.js module used for WebGL rendering and 3D scene construction.                                                                        |
| <code>GLTFLoader.js</code>                               | Loads the <code>.glb</code> models (the elven character, NPCs, trees, rocks and other props).                                                   |
| <code>OBJLoader.js</code> /<br><code>MTLLoader.js</code> | Load the <code>.obj</code> building models together with their <code>.mtl</code> materials.                                                     |
| <code>RoomEnvironment.js</code>                          | Generates a neutral image-based environment used (via a PMREM generator) to give physically based materials soft reflections and ambient light. |
| Browser WebGL<br>implementation                          | GPU-backed rendering target used by Three.js through <code>THREE.WebGLRenderer</code> .                                                         |

Three.js and its addons are loaded from a CDN through an `importmap` declared in `index.html`, which maps the bare module name `three` (and the `three/addons/` path) to the corresponding CDN URLs.

### Models and assets

External 3D models (trees, rocks, character meshes, building models) are loaded from `.glb` and `.obj` files and are placed and animated in code. Several decorative textures — such as banner cloth, market stall details, and terrain patches — are generated procedurally with the HTML canvas and uploaded as `THREE.CanvasTexture`.

### 3 Application Architecture

The project is a static browser application organized into focused ES modules. The main files are:

| File                                                                                                                                                                                                                                                                        | Purpose                                                                                                                                                      |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>index.html</code>                                                                                                                                                                                                                                                     | Declares the canvas, main menu, loading screen, HUD container and the import map for Three.js.                                                               |
| <code>main2.js</code>                                                                                                                                                                                                                                                       | Entry point: renderer and camera setup, scene construction, environment lighting, assembly of all world modules, the animation loop and the day/night cycle. |
| <code>elfWarrior.js</code>                                                                                                                                                                                                                                                  | Loads and controls the playable character: input, third-person follow camera, procedural walk/idle animation and movement.                                   |
| <code>npcManager.js</code> ,<br><code>dialogueSystem.js</code> ,<br><code>questSystem.js</code> ,<br><code>inventory.js</code> ,<br><code>collectibles.js</code> ,<br><code>hud.js</code>                                                                                   | Gameplay systems: wandering NPCs, dialogue, quests, inventory, herb gathering and the on-screen HUD.                                                         |
| World modules<br>( <code>windmill.js</code> ,<br><code>well.js</code> ,<br><code>buildings.js</code> ,<br><code>trees.js</code> , <code>walls.js</code> ,<br><code>stalls.js</code> , <code>fields</code> ,<br><code>lights.js</code> ,<br><code>lampPosts.js</code> , ...) | Each constructs one type of world object or feature and returns it to <code>main2.js</code> .                                                                |
| <code>style.css</code>                                                                                                                                                                                                                                                      | Styles the full-screen canvas, menu, loading screen and HUD.                                                                                                 |

#### Scene startup

At startup, `main2.js` creates the renderer, the perspective camera, the scene, the environment map and the lights, then builds the terrain and every world object, loads the character and the NPCs, and starts the animation loop. The renderer uses anti-aliasing, tone mapping, and PCF soft shadow maps. The camera is a `THREE.PerspectiveCamera`, so distant objects appear smaller and the user sees a natural 3D projection.

#### Scene graph organization

The application uses the Three.js scene graph extensively. A parent group can contain several child meshes and child groups, so a complete object can be moved or rotated by transforming only its parent. For example, the windmill group contains a rotating `blades` group, which in turn contains four sail `arm` groups, each holding its spar, sail and slats.

#### Update loop

The animation loop is driven by `requestAnimationFrame`. Every frame updates elapsed time, character input and movement, the follow camera, NPC wandering and gestures, the windmill rotation, animated water, and the day/night lighting, then renders the scene.

### 4 Hierarchical Modeling

Hierarchical modeling is one of the central technical parts of the project. Instead of treating every mesh as an unrelated object, the project uses nested groups and skeletons.

### Player character hierarchy

The main example of hierarchical modeling in the project is the playable elf controlled by the user. It is a skinned mesh loaded from a `.glb` file, deformed by an internal *skeleton* of bones arranged in a parent-child chain. A top-level `THREE.Group` (the character root) positions and rotates the whole elf in the world, while the bones inside the skeleton control the body. The simplified structure is:

```
elf root (Group)          <- world position + heading
  skinned mesh
    Pelvis / Waist        <- torso twist during walking
    Spine01               <- subtle breathing while idle
      L_Upperarm -> L_Forearm
      R_Upperarm -> R_Forearm
    L_Thigh               -> L_Calf
    R_Thigh               -> R_Calf
```

Because the bones form a hierarchy, rotating a parent bone automatically moves every child attached to it: turning a thigh carries its calf (and the foot) with it, and turning an upper arm carries the forearm. The animation never sets world positions of individual limbs by hand; it only sets joint rotations at meaningful pivots and Three.js combines the transforms down the chain.

The character's motion is fully procedural and built on this hierarchy. During walking, the left and right `Thigh` bones swing in phase opposition driven by a sine wave whose speed follows the character's velocity; the `Calf` bones flex on the recovery part of each step; the arms swing in counter-phase to the legs; and the `Waist` adds a slight torso twist while the whole mesh bobs vertically. When idle, a slow sine wave on `Spine01` and the upper arms produces a gentle breathing motion and during a jump the thighs and calves tuck according to the vertical velocity.

### NPC hierarchy

The NPCs are built the same way — skinned meshes with their own skeletons, wrapped in a root group — and a gesture controller applies small procedural rotations to individual arm bones so they do not look static as they wander.

### Grouped world models

The same principle extends to non-character objects assembled from nested `THREE.Groups`. The windmill is a three-level hierarchy (a `blades` group holding four `sail arm` groups, each with its `spar`, `sail`, and `slats`) so that rotating only the `blades` group turns all four sails together. The well nests a `crank` group and a `swing` group (with a `bucket` sub-group) inside the main `well` group, so the bucket and rope sway as one assembly.

### Why this matters

The hierarchy makes animation simpler and more correct: instead of recomputing every child position by hand, the project changes rotations at meaningful pivots and lets Three.js combine the transforms automatically through the parent-child chain.

## 5 Geometry, Meshes, Textures, and Materials

Many world objects are built from Three.js primitive geometry — boxes, cylinders, spheres, cones, planes — combined into larger models using groups. Larger imported assets (character, trees, rocks, buildings) are loaded as `.glb` or `.obj` files and integrated into the same scene graph.

### Textures

The project mixes two sources of textures. Detailed surfaces such as brickwork and roof tiles use loaded image maps (base color, roughness and normal maps) applied through `THREE.TextureLoader`.

Other decorative surfaces are generated in JavaScript with the HTML canvas and uploaded as `THREE.CanvasTexture`.

| Texture type             | Example in the project                                                                |
|--------------------------|---------------------------------------------------------------------------------------|
| Colour / base map        | Brick walls, roof tiles, banner cloth, market stall details, and terrain patches.     |
| Normal map               | Brick and stone surfaces use normal maps for fine relief without extra geometry.      |
| Roughness map            | Surfaces vary their glossiness through roughness maps.                                |
| Ambient occlusion / bump | Selected materials use AO and bump maps to deepen contact shadows and surface detail. |
| Procedural (canvas)      | Banner cloth, stall fabrics and terrain patches are drawn at runtime on a canvas.     |

## Materials

The project relies mainly on `MeshStandardMaterial` for physically based surfaces (stone, wood, cloth, terrain, foliage, emissive elements). `MeshPhysicalMaterial` is used for the water of the lake, where transmission, clearcoat and a realistic index of refraction give a convincing liquid surface (with transmission enabled only on the high-quality tier). A few simple unlit elements use `MeshBasicMaterial`. Physically based parameters such as metalness, roughness, and emissive intensity are tuned so the world stays readable across the full day/night cycle.

## 6 Lighting, Shadows, Cameras and Shading

The scene combines several light contributions to create a believable outdoor environment that stays readable from many positions.

### Lighting setup

- **Hemisphere light:** provides broad ambient sky and ground illumination.
- **Directional light:** acts as the main sun light and casts the scene's large shadows.
- **Secondary directional light:** a soft fill light that lifts shadowed areas.
- **Emissive lamp posts:** the lamp posts carry light sources that switch on at dusk, lighting the streets at night.
- **Environment map:** a `RoomEnvironment` processed through a PMREM generator gives materials soft image-based ambient light and reflections.

The renderer enables `THREE.PCFSoftShadowMap`. Many meshes cast and receive shadows, which grounds objects on the terrain and improves depth perception. The **day/night cycle** runs continuously over a fixed period: it changes light intensities and colors and, in the evening, turns on the lamp posts, shifting the mood of the whole village.

### Camera and projection

The main camera is a `THREE.PerspectiveCamera`. Rather than orbiting the world from the outside, it follows the character in third person: the mouse rotates the camera around the character (yaw and pitch), the scroll wheel changes the follow distance, and the camera direction also defines the character's forward movement. This view-relative scheme replaces the default `OrbitControls` with a control style suited to an explorable game world.

## 7 Animation

All character and world animation is implemented in JavaScript and driven by elapsed time, state variables, interpolation and procedural updates.

### Character animation

The character's motion is procedural. A walk cycle drives the legs and arms through a phase value that advances with movement speed, a rest pose keeps the arms naturally at the sides when idle and a jump applies vertical velocity with gravity. The follow camera smoothly interpolates toward its desired position each frame so motion stays fluid.

### NPC animation

NPCs wander autonomously inside a radius around their home position: they pick a target, walk toward it, pause, and choose a new one. A gesture controller applies small procedural arm and body motion so the characters do not look static, with per-NPC options that respect fixed arm poses where needed.

### World animation

The windmill sails rotate continuously by advancing the blades group rotation. The water surface animates over time. The day/night cycle is itself an animation, smoothly interpolating lighting and lamp-post emission across the full scene.

## 8 Implemented User Interactions

The project is both an explorable scene and a small game. Controls are bound to state in JavaScript and to keyboard and mouse input.

### Movement (third person)

- W / Up: move forward.
- S / Down: move backward.
- A / Left: strafe or turn left.
- D / Right: strafe or turn right.
- Shift: run faster.
- Space: jump.

### Camera

- Mouse drag: orbit the follow camera around the character.
- Mouse wheel: zoom the follow camera in and out.

### Gameplay interaction

- E: talk to the nearest NPC (proximity based).
- F: gather a nearby herb / collectible.
- I: open or close the inventory.
- Esc: close the active dialogue.

An on-screen HUD shows a framed character portrait and a health bar. A main menu and an animated loading screen with a progress bar precede the playable scene.

## 9 How to Run, Deploy, and Test

### Run locally

From the project root, start a static web server:

```
python -m http.server 8000
```

Then open `http://localhost:8000`. A local server is required because the project uses JavaScript modules and an import map; opening the HTML file directly from disk may fail in some browsers because module loading is stricter than normal scripts.

### GitHub Pages deployment

The project runs on GitHub Pages: all asset paths are relative, and Three.js is loaded from a CDN through the import map. After pushing to the class repository, Pages is enabled from the repository settings using the `main` branch and the root directory. The live URL is:

```
https://sapienzainteractivegraphicscourse.github.io/  
final-project-eldermoor_dev/
```

### Performance notes

The scene is real-time and avoids expensive offline techniques. Large models were compressed to reduce download size, foliage and props are reused across the map, and shadows use PCF soft shadow maps to balance quality and cost.

## 10 Known Scope Limits and Contributions

### Known scope limits

The project uses real-time WebGL rasterization. It does not implement a full ray tracer, path tracer, global illumination solver, or a rigid-body physics engine; collisions are handled by a lightweight, custom collision system rather than a physics simulation. The techniques used are:

- PCF shadow maps for real-time shadows,
- PBR-style material parameters with an image-based environment,
- a mix of loaded image textures and procedural canvas textures,
- a procedural, JavaScript-authored animation system for the character, NPCs and world objects,
- a lightweight custom collision system that keeps the player within the walkable area using 2D primitives on the ground plane,
- a continuous day/night lighting cycle.

This scope keeps the project interactive and reliable in the browser while still demonstrating the core graphics concepts.

### Contributions

This project was designed and implemented individually, covering the environment and world construction, the hierarchical character and NPC system, the procedural animation, the lighting and day/night cycle, the texture and material setup and the gameplay interactions and UI.

### Conclusion

Eldermoor is a complete interactive graphics project. It combines hierarchical modeling, transformations, perspective cameras, WebGL rasterization, meshes, textures, materials, lights, shadows, procedural animation, and user interaction in a single explorable browser scene — both a technical demonstration and a playable 3D world.